.

DocuGen — Project Documentation

***Source**: https://github.com/jaswanthg1040/DocuGen.git*

Prototype: https://docugen-jaswanthg1040.streamlit.app/

INDEX

CONTENTS	PAGE
1. Abstract	3
2. Overview	3
3. Features	4
4. Architecture	5
5. Data Model	7
6. Formatting Syntax	7
7. Installing and Running Locally	8
8. Using the Application	9
9. Streamlit Prototype and Deployment	10
10. Testing	11
11. Security and Privacy	11
12. Known Limitations	12
13. Troubleshooting	12
14. Extending the Project	13
15. Appendix	14

1. ABSTRACT

DocuGen – AI Document Studio is a Python-based web application designed to simplify the process of creating, editing, managing, previewing, importing, and exporting digital documents through a user-friendly interface. The application is developed using Streamlit for the frontend, SQLite for database management, python-docx for Word document generation, and ReportLab for PDF generation.

The system provides essential document management features, including a dashboard, document editor, predefined templates, search and filtering, automatic draft recovery, version history, and multiple export formats such as DOCX, PDF, Markdown, and plain text. It also supports light and dark themes, customizable export settings, document organization, and database backup functionality.

DocuGen aims to provide a simple, efficient, and reliable document management solution without requiring paid services or external API keys. Its modular architecture, automated testing, and local database storage make it suitable for personal and academic document management.

Keywords: DocuGen, Document Management, Python, Streamlit, SQLite, DOCX, PDF, Document Editor, Automation.

2. Overview

DocuGen is a web application for creating, editing, previewing, saving, importing and exporting documents. It is written in Python and uses Streamlit for the interface, SQLite for storage, python-docx for Word files and ReportLab for PDF files. It was designed as a **single-user, local** tool that needs no paid service or API key.

Item	Detail
Product name	DocuGen — AI Document Studio
Version	2.0.0 (defined in app.py)
Repository	github.com/jaswanthg1040/DocuGen, one commit: “Update config.toml” (8ec36be)
Live prototype	docugen-jaswanthg1040.streamlit.app
Language and size	Python 3.10+, about 3,400 lines including 97 automated tests
Licence of bundled font	DejaVu Sans, free licence (see fonts/LICENSE-DejaVu.txt)

How this document was verified

- **Repository:** cloned and read in full. All **97 automated tests passed** on the cloned code.
- **Live prototype:** *not* inspected. The site could not be reached from the documentation environment, so nothing here is a claim about how the deployed instance currently behaves. Section 8 lists what to check on it.
- **AI features:** the product name contains "AI", but AI-assisted writing is **not implemented**. The Settings page says so. Every feature works without an AI service.

3. Features

Area	What it does
Dashboard	Welcome page, quick actions, live statistics (saved documents, modified this week, favorites, templates), five most recent documents
Editor	Title, description, tags, Markdown-style text box, Write/Preview tabs, quick-insert buttons, word and character count, reading time, find and replace
Save and safety	Save and update, unsaved-change prompts, autosaved draft with recovery, version history (last 20 versions per document)
My Documents	Search by title, four sort orders, tag filter, favorites, Open, Edit, Export, Copy, Pin, Delete with confirmation
Templates	Blank, Project Report, Meeting Minutes, Business Proposal, Professional Letter, Study Notes, Resume/CV, Research Report
Import	.docx, .txt, .md, .markdown up to 5 MB
Export	Word (.docx), PDF, Markdown, plain text; A4 or Letter; optional page numbers
Settings	Light/Dark theme, default export format, page size, Word font, description in exports, page numbers, database backup download

4. Architecture

4.1 Technology stack

Layer	Technology	Purpose
Interface	Streamlit 1.40+ with custom CSS	Pages, widgets, session state, theme
Storage	SQLite (Python sqlite3)	Documents, versions, draft, settings
Word export	python-docx 1.1+	Valid `.docx` files
PDF export	ReportLab 4+	Valid PDF files
Configuration	python-dotenv	Optional `.env` overrides
Testing	pytest 8+ and Streamlit AppTest	Unit and app-level tests

4.2 Layers and modules

The code separates the interface, the data layer and the export logic so that each can change independently.

File	Responsibility
`app.py`	All pages, navigation, session state, callbacks, autosave, export buttons
database.py	Every SQL statement, input validation, schema creation and upgrade
`components/ui.py`	Light and dark theme CSS, hero, cards, badges, empty states
services/formatting.py	Parser for the supported syntax, plus word count and preview conversion
`services/word_export.py`	Builds `.docx` bytes
services/pdf_export.py	Builds PDF bytes and registers fonts
`services/text_export.py`	Markdown and plain-text output
services/importer.py	Converts uploaded files to the supported syntax

File	Responsibility
services/templates.py	The eight starter templates, stored as application data
services/utils.py	Safe filename generation

4.3 Key design decision: one parser for everything

services/formatting.py turns text into blocks (heading, paragraph, bullet, number, table, rule). The preview, the Word export, the PDF export, the text export and the word counter all use these same blocks. This keeps every output consistent and means a new formatting feature is added in one place.

4.4 Navigation and session state

- The sidebar is a Streamlit radio bound to nav_page. Code never changes it directly; it sets _goto, and main() applies it before the sidebar is drawn. This avoids Streamlit's restriction on changing a widget's state after it is drawn.
- The editor holds its text in a draft dictionary, copied from and back to widgets on each run. Widget keys are cleared whenever a different document is loaded, so text cannot leak from one document into another.
- A snapshot of the last saved values decides whether the editor is "dirty". Opening a template, document or imported file over unsaved text always asks for confirmation first.
- Buttons use callbacks, and messages are passed to the next run through a flash entry.

4.5 Autosave and recovery

On every run where the editor is dirty, the draft is written to a single-row table. A new session that finds a draft shows a banner on the Dashboard and Editor with **Restore draft** and **Discard it**. Saving or discarding clears the draft. Only the most recent draft is kept.

5. Data model

The database file is `data/docugen.db`, created automatically. Set `DOCUGEN_DB_PATH` in a `.env` file to move it. Databases created by version 1.x gain the new columns automatically.

Table	Columns	Notes
`documents`	id, title, description, content, created_at, updated_at, pinned, tags	Tags are stored lower-case, comma-separated, at most 10 per document and 30 characters each
versions	id, doc_id, title, description, content, saved_at	A snapshot of the previous text is stored on each real change; the newest 20 are kept; deleted with the document
`draft`	id (always 1), doc_id, title, description, tags, content, saved_at	The single autosaved draft
settings	key, value	Theme, default export, Word font, page size, page numbers, description toggle

Limits: title 200 characters, description 500, content 1,000,000. All queries use parameters, and sort order comes from a fixed whitelist.

6. Formatting syntax

The editor is a plain-text box. The Preview tab and every export understand the following.

You type	Result
`# Heading`, `## Heading`, `### Heading`	Heading levels 1 to 3
- item (also * or +)	Bullet; indent by 2 spaces per level, up to 3 levels
`1. item`	Numbered item; numbering restarts after a paragraph or heading
bold, *italic*, ***both***	Inline formatting
Backslash then asterisk	A literal asterisk
| A | B | lines	Table; the first row is the header and a separator row is optional

You type	Result
`---` on its own line	Horizontal rule
Blank line	New paragraph; a single line break inside a paragraph is kept

Not supported: images, links, colours, alignment, merged cells, nested tables.

7. Installing and running locally (Windows 11)

7.1 One-click launcher

Install Python 3.10 or newer and tick **Add python.exe to PATH**. Unzip or clone the project and double-click `Start DocuGen.bat`. On first run it creates `.venv`, installs the dependencies (internet required) and starts the app. The launcher could not be executed in the Linux build environment, so test it on your own PC.

7.2 Manual setup in PowerShell

Run these commands in the project folder:

1. `python -m venv .venv`
2. `.\.venv\Scripts\Activate.ps1`
3. `python -m pip install --upgrade pip`
4. `python -m pip install -r requirements.txt`
5. `python -m streamlit run app.py`

If activation is blocked, run `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned` once.

7.3 Opening the app

Open **http://localhost:8501** in a browser. **Repository note:** `.streamlit/config.toml` in the repository sets `headless = true`, so Streamlit will *not* open a browser tab automatically, although the README says it does. Either open the address yourself or set `headless = false`.

7.4 Optional configuration

Copy `.env.example` to `.env` and uncomment `DOCUGEN_DB_PATH` to store the database elsewhere. A relative path is resolved from the project folder.

8. Using the application

Page	Typical tasks
Dashboard	Start a document, open saved documents, browse templates, import a file, restore a recovered draft
Create Document	Write and preview, add tags, insert a table or list, find and replace, save, export, restore an earlier version
My Documents	Search, sort, filter by tag, show favorites only, open a preview, edit, export, copy, pin, delete
Templates	Pick a starting structure; nothing is saved until you click **Save document**
Settings	Theme, export defaults, page size, fonts, backup download
Help & About	Short in-app guide

8.1 Creating and saving a document

1. Choose **Create Document**, a template, or **Import a file**.
2. Type a title (required) and the content.
3. Click outside the text box or press Ctrl+Enter so the Preview tab refreshes.
4. Click **Save document**. Saving again updates the same document and stores the previous text in Version history.
5. Use **New document** to start another one; it never overwrites the open one.

8.2 Exporting

The Export area offers Word, PDF, Markdown and Text downloads. They use the text currently in the editor, even if it is unsaved. The default format in Settings is highlighted. File names are sanitised for Windows, macOS and Linux.

8.3 Recovering work

- **After a refresh or crash:** use the Restore draft banner.
- **Earlier text of a saved document:** open it, expand **Version history** and click **Restore**. The version loads as an unsaved change, so nothing is lost until you save.

9. The Streamlit prototype and deployment

The prototype at docugen-jaswanthg1040.streamlit.app appears to be the same application hosted on Streamlit Community Cloud. Because DocuGen was built as a private, local tool, hosting it publicly changes several things. These points follow from how the code works and how that platform generally behaves, and have **not** been confirmed against the live instance.

Topic	What to expect	Recommendation
Shared data	There is no login and one database file, so every visitor shares the same documents, drafts and settings	Treat the prototype as a demo; do not enter private information. Add authentication before real use
Autosave and recovery	There is only one draft row, so two visitors can see each other's recovered draft	Same as above
Persistence	The server file system is generally temporary, so documents may disappear after a restart or redeploy	Use the backup download, or move storage to an external database
Theme	The theme is stored in the shared database, so one visitor's change affects all	Move preferences into per-session or per-user storage
Fonts	The PDF font is bundled in fonts/, so PDFs do not depend on server fonts	None needed
Dependencies	requirements.txt is the only file the host needs	Keep version ranges current

9.1 Checklist for verifying the live prototype

1. Open the Dashboard and confirm the welcome text and four statistic cards appear.
2. Create a document from the Project Report template, save it and refresh. It should still be listed.
3. Type unsaved text, refresh and look for the recovery banner.
4. Export all four formats and open the Word and PDF files.
5. Switch to Dark in Settings and confirm the whole interface changes.
6. Open the app in a private window and note whether documents from the first window are visible. If they are, the data is shared.

10. Testing

Run `python -m pytest -q` in the project folder. Result on the cloned repository: **97 passed**.

Test file	Focus
`test_database.py`, `test_v2_database.py`	Create, read, update, delete, search, sorting, persistence, upgrade from 1.x, versions, tags, favorites, drafts, settings, unwritable location
`test_exports.py`, `test_v2_formatting_and_files.py`	Word and PDF validity, structure and formatting, empty and very long documents, Unicode, filenames, tables, nested lists, page size, text exports, importer including a Word round trip, rejection of bad files
`test_app.py`, `test_v2_app.py`	Syntax and imports of every file, all pages render, template to editor to save, unsaved-change prompts, search and delete confirmation, theme and settings persistence, recovery after a simulated restart, version restore, pin, copy, tag filter, find and replace

The app tests use Streamlit's headless AppTest runner, not a real browser, so visual appearance, including dark mode, is not covered by automated tests.

11. Security and privacy

- All SQL is parameterised. Search text is escaped so percent and underscore are literal.
- Document text is never rendered as HTML. The preview escapes anything outside the supported syntax, and titles shown in cards are escaped.
- Imported files are parsed, never executed, and limited to 5 MB.
- Exports remove control characters that would corrupt Word or PDF files.
- The database is **not encrypted** and there is **no authentication**. Anyone who can reach the app or the data folder can read every document.
- No API keys are used or stored.

12. Known limitations

- AI-assisted writing is not implemented.
- The editor is plain text with Markdown-style syntax, not WYSIWYG.
- PDF output supports Western European, Greek and Cyrillic text. Chinese, Japanese, Korean, Telugu and other Indic scripts, Arabic and emoji do not render correctly in PDFs. Word, Markdown and text exports keep all Unicode text.
- Word numbered lists use typed numbers with a hanging indent rather than Word's automatic numbering.
- Importing a Word file keeps headings, lists, tables, bold and italic only; images, links, comments and tracked changes are dropped.
- Only one autosaved draft is kept.
- Dark mode is custom CSS and may need updating after major Streamlit releases.
- The preview approximates the exported layout and refreshes when the text box loses focus.

13. Troubleshooting

Problem	Fix
`python` is not recognised	Install Python and tick Add python.exe to PATH, or use `py -m venv .venv`
No module named streamlit, docx or reportlab	Activate .venv and run python -m pip install -r requirements.txt
Browser does not open	Open http://localhost:8501 manually (`headless = true` in the repository)
File does not exist: app.py	Change into the folder that contains app.py
Port 8501 already in use	Add `--server.port 8502` to the run command
Cannot open its database	Make the data folder writable, or set DOCUGEN_DB_PATH to a writable path
SyntaxError or IndentationError	A file was edited incorrectly; restore it from the repository and rerun the tests
PDF shows empty boxes	The text uses a script the PDF font lacks; use the Word export

14. Extending the project

Goal	Where to change
Add a template	Append an entry to `TEMPLATES` in `services/templates.py` (id, icon, name, description, title, content); the gallery and the dashboard count update automatically
Add a formatting feature	Teach `parse_blocks` in `services/formatting.py`, then handle the new block kind in the preview, Word, PDF and text exporters, and add tests
Add an export format	Create a function in `services/`, then add a download button in `export_buttons` in `app.py`
Add a setting	Add a default in `DEFAULT_SETTINGS`, validate it in `load_settings`, and add a widget with `cb_update_setting` in `page_settings`
Add AI assistance	Create a provider module that reads credentials from environment variables, and show its buttons only when configured; keep core features independent of it
Add users	Add authentication and a `user_id` column on documents, versions and draft, then filter every query in `database.py` by it

15. Appendix

15.1 Dependencies (`requirements.txt`)

`streamlit>=1.40,<2` · `python-docx>=1.1,<2` · `reportlab>=4.0,<5` · `python-dotenv>=1.0,<2` · `pytest>=8,<10`

15.2 Repository layout

`Start DocuGen.bat` · `app.py` · `database.py` · `requirements.txt` · `pytest.ini` · `.env.example` · `.gitignore` · `README.md` · `.streamlit/config.toml` · `components/ui.py` · `services/` (formatting, word_export, pdf_export, text_export, importer, templates, utils) · `fonts/` (DejaVu Sans, four files and licence) · `data/` (database created here, ignored by Git) · `tests/` (six test files)

15.3 Glossary

Term	Meaning
Draft	The unsaved editor text, autosaved for recovery
Version	An earlier saved state of a document
Block	One parsed unit of text such as a heading, list item or table
Pinned / favorite	A document shown at the top of My Documents
Headless	Streamlit setting that stops the browser opening automatically